

Bootloading Procedure for Atmel® Touch Sensors Based on the Object Protocol

1. Introduction

This document describes how to update the firmware on Atmel touch sensor controllers that are based on the Object Protocol.

Note: After updating the firmware the host should rewrite any stored settings used by the application.

2. Entering the Bootloader Mode

2.1 Introduction

Bootloader mode is entered automatically on power-up if the firmware on the device is missing or corrupt.

If the firmware is valid and the device is running in application mode, it can be put into bootloader mode using one of the three force-flash sequences:

- Writing 0xA5 to the command processor object's $\overline{\text{RESET}}$ field (see [Section 2.2](#))
- Using the $\overline{\text{CHG}}$ and $\overline{\text{RESET}}$ line together (see [Section 2.3](#))
- Toggling the $\overline{\text{RESET}}$ line (see [Section 2.4](#))

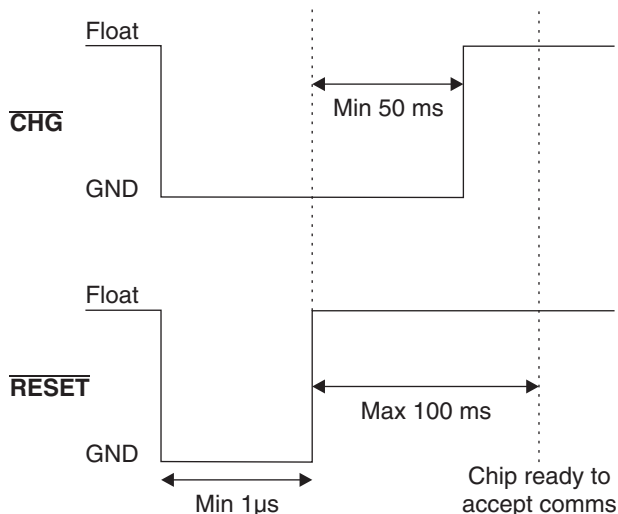
2.2 Command Processor Force-flash Sequence

Write 0xA5 to the Command Processor Object's RESET field to enter the bootloader mode. Refer to the *Protocol Guide* for your device for information on how to do this.

2.3 $\overline{\text{CHG}}$ and $\overline{\text{RESET}}$ Force-flash Sequence

With this sequence the $\overline{\text{CHG}}$ line is held low while the chip is powered up after a reset (see [Figure 2-1](#)).

Figure 2-1. $\overline{\text{CHG}}$ and $\overline{\text{RESET}}$ Force-flash Sequence



Bootloading Procedure

Application Note QTAN0051

**REFLASH
AGREEMENT
REQUIRED**

maXTouch™

10725BX-AT42-03/10

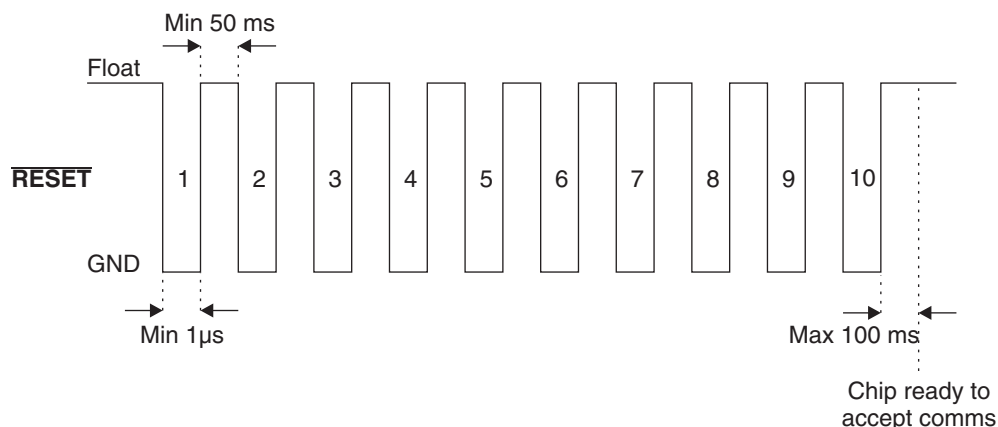


FOR RELEASE ONLY UNDER NON-DISCLOSURE AGREEMENT (NDA)
AND ONCE A LEGAL REFLASH AGREEMENT HAS BEEN EXECUTED

2.4 $\overline{\text{RESET}}$ Toggling Force-flash Sequence

With this sequence the $\overline{\text{RESET}}$ line is asserted ten times in a row without communicating via the I²C-compatible bus between the resets.

Figure 2-2. $\overline{\text{RESET}}$ Toggling Force-flash Sequence



2.5 Confirmation of Bootloader Mode

When it receives a reset command, the device resets itself and starts up in bootloader mode.

The I²C-compatible slave address is different to that used in application mode (see [Table 3-1 on page 3](#)).

The host can confirm that the device is in bootloader mode (WAITING_BOOTLOAD_CMD state; see [Table 3-3 on page 5](#)) by reading a status byte from the bootloader address. If the read is successful, bits 6 and 7 of the status byte returned will be set, and the other 6 bits will contain the bootloader version (see WAITING_BOOTLOAD_CMD state in [Table 3-3 on page 5](#)).

In application mode, if no application is programmed, or the application is corrupt, the $\overline{\text{CHG}}$ line will be set low. The host needs to perform an I²C-compatible read to retrieve the error code and set the device into bootloader mode.

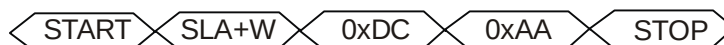
3. Updating the Application

3.1 Unlocking the Device

The device first needs to be placed in bootloader mode using one of the force-flash sequences described in [Section 2](#) to unlock the device and start bootloading a new application.

Once the device is in bootloader mode, the host needs to write the Application Update Unlock Command (0xDC followed by 0xAA) to the device to unlock it (see [Figure 3-1](#)).

Figure 3-1. Application Update Unlock Command



3.2 Application Update Mode

3.2.1 Introduction

In application update mode, the device accepts multibyte update commands when written to over the I²C-compatible interface, and returns a single status byte when it is read.

The device responds to the I²C-compatible address selected by the address pins. See [Section 3.2.3](#) and [Section 3.2.4](#) for more details.

3.2.2 $\overline{\text{CHG}}$ line

The device indicates a transition between the states by asserting the $\overline{\text{CHG}}$ line (that is, by setting it low). Once the status byte has been read, the device releases the $\overline{\text{CHG}}$ line (that is, the device floats the $\overline{\text{CHG}}$ line and then the pullup returns it high). It then runs the processes associated with that state. The device waits until the status is read, so that the host cannot miss a state transition.

Note: It is very important that the host checks the status of the $\overline{\text{CHG}}$ line in order to know when it needs to read the device. The host should not simply poll the I²C-compatible bus, as I²C-compatible reads or writes before the $\overline{\text{CHG}}$ line is asserted are not supported.

3.2.3 Single Device Addresses

In the case of a single maXTouch device (such as the mXT224, mXT165 and mXT140) the device responds to the I²C-compatible address selected by the ADDR_SEL pin (see [Table 3-1](#)).

Table 3-1. I²C-compatible Address Selection

ADDR_SEL	Mode	I ² C-compatible Address
Low	Application	0x4A
High		0x4B
Low	Bootloader	0x24
High		0x25

3.2.4 Chip Set Device Addresses

In the case of maXTouch chip set devices (such as the mXT1386 Chip Set) the device responds to the I²C-compatible address selected by the A1 and A0 pins (see [Table 3-2](#)).

Table 3-2. I²C-compatible Address Selection

A1	A0	Mode	I ² C-compatible Address
0	0	Application	0x4C
0	1		0x4D
1	0		0x5A
1	1		0x5B
0	0	Bootloader	0x26
0	1		0x27
1	0		0x34
1	1		0x35

3.2.5 State Machine

The application update mode works as a state machine as shown in Figure 3-2 on page 4.

Figure 3-2. Application Update Mode Program Flow

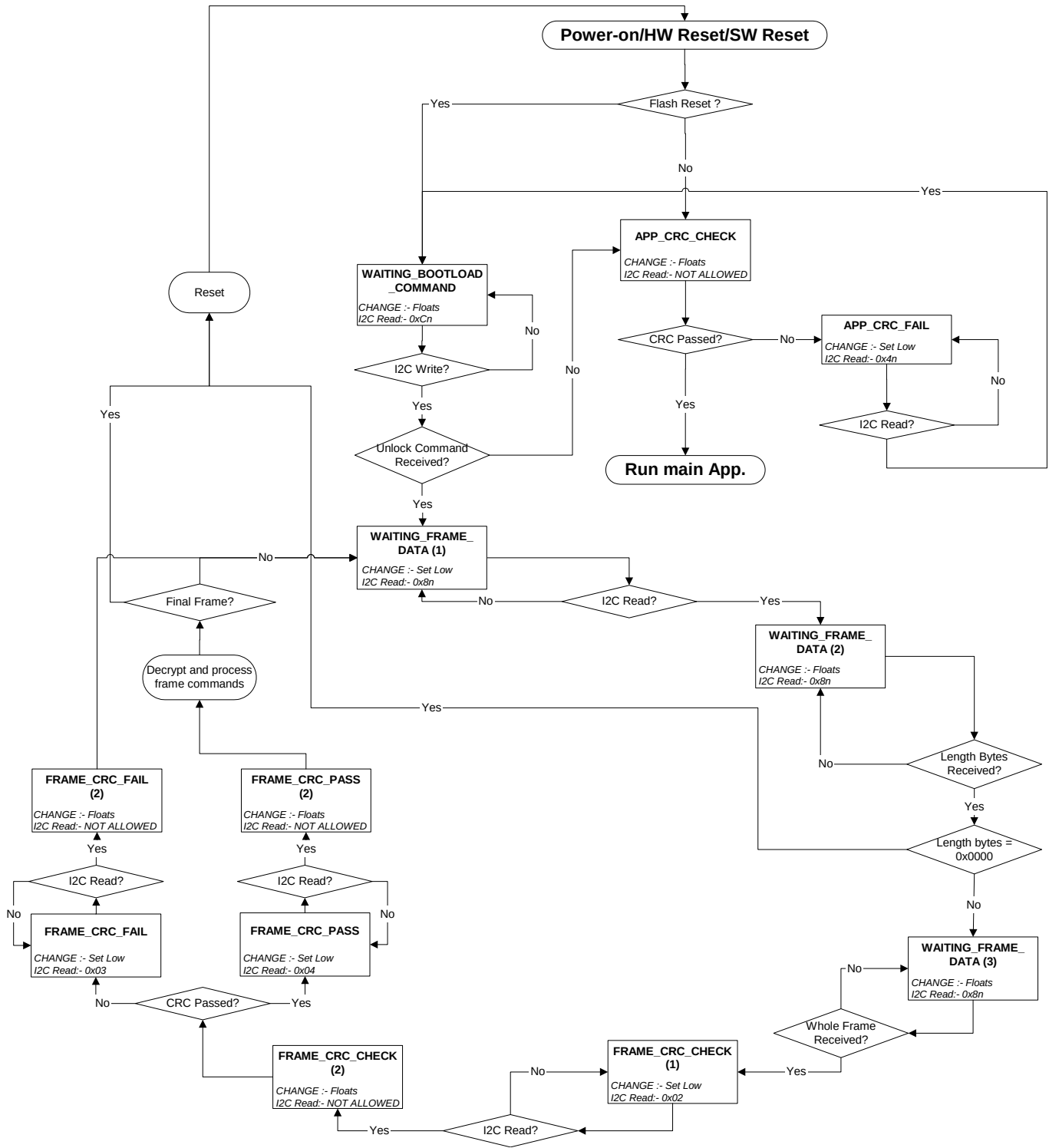


Table 3-3 describes the behavior of each state.

Table 3-3. Application Update Mode Status Codes

State	Data Transmission	Description
0b11nnnnnn	Yes	WAITING_BOOTLOAD_CMD: The data returned by the device has bit 7 and 6 set. The remaining 6 least significant bits are part dependent and provide information about the bootloader version (see Section 3.3 on page 6). In this state, the device is awaiting the bootloader command. Sending any other byte than the bootloader command causes the device to leave bootloader mode.
0b10nnnnnn	Yes	WAITING_FRAME_DATA: The data returned by the QT device has bit 7 set. The remaining 6 least significant bits are part dependent and provide information about the bootloader version (see Section 3.3 on page 6). In this state, the device is awaiting frame data from the host. Once it has received a whole frame it moves to the CRC check state.
0b00000010	No	FRAME_CRC_CHECK: The host does nothing at this stage but waits for the CRC check result.
0b00000011	No	FRAME_CRC_FAIL: If the state machine reaches this state, the CRC check for the current frame has failed. The host should try resending the last frame once the device returns to the WAITING_FRAME_DATA state.
0b00000100	No	FRAME_CRC_PASS: If the state machine reaches this state, the current frame data has passed the CRC check, and the device is proceeding to process the frame records. The host should wait until the device returns to the WAITING_FRAME_DATA state before sending the next frame.
0b01nnnnnn	No	APP_CRC_FAIL: The data returned by the device has bit 6 set. The remaining 6 least significant bits are part dependent and provide information about the bootloader version (see Section 3.3 on page 6). If the state machine reaches this state, the CRC check for the currently stored application code failed. The host can make the device enter bootloader mode to recover the firmware by sending the bootloader command sequence.

3.3 Bootloader Version Information

Some of the states return the bootloader version information within their status codes. [Table 3-4](#) shows how to decode this information.

Table 3-4. Bootloader Version Information

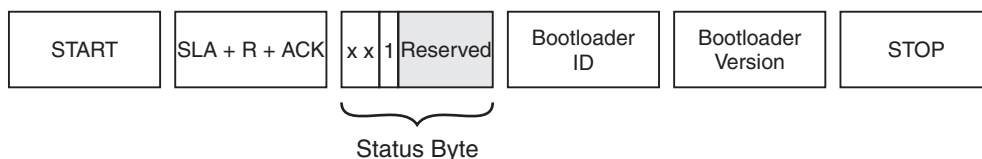
Status Byte				Description
Bit 7	Bit 6	Bit 5	Bits 4..0	
x	x	0	Bootloader ID	Bit 5 of the status byte is set to 0: Bits 0..4 contain a 5-bit bootloader ID for the device (for details of the ID, refer to the <i>Protocol guide</i> for your device).
x	x	1	Reserved	Bit 5 of the status byte is set to 1: The bootloader ID is read by reading two additional bytes. See Figure 3-3 .

To read the bootloader ID from a bootloader that uses the extended ID mode, a 3-byte I²C-compatible read must first be performed (see [Figure 3-3](#)):

1. The first byte is the status byte with bit 5 set to 1 to indicate extended ID mode.
2. The second byte is the bootloader ID code.
3. The third byte is the bootloader version code.

Note that this three-byte read is necessary only once to obtain the bootloader ID. The rest of the application update process may be performed using single-byte reads.

Figure 3-3. Extended ID Mode



3.4 Data Frames

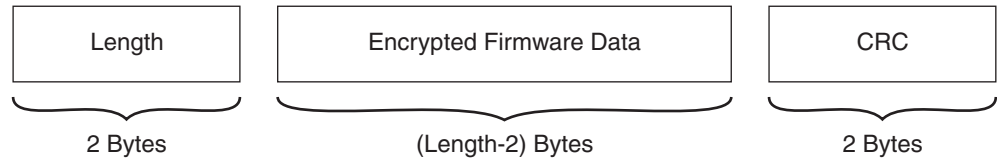
The application firmware is sent to the device as frames of data. Each frame (see [Figure 3-4](#)) consists of:

- A 2-byte length field, detailing the number of bytes to follow in the frame
- The encrypted firmware data
- A 2-byte CRC

The maximum size of a frame depends on the type of device:

- For single maXTouch devices (such as the mXT224, mXT165 and mXT140) the maximum size of a frame is 276 bytes.
- For maXTouch chip set devices (such as the mXT1386 Chip Set) the maximum size of a frame is 532 bytes.

Figure 3-4. Data Frame Format

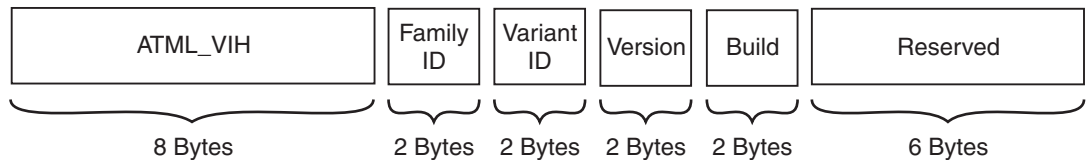


The application frames come from a file supplied by Atmel. This is one of two formats:

- A file containing only the frame data (see [Figure 3-4](#))
- A file containing the frame data and header information (see [Figure 3-5](#))

A file that contains header information starts with the characters ATML_VIH, and consists of a total of 22 characters. The header information will describe the contents of the encrypted firmware file and will be in the following format.

Figure 3-5. Encrypted Firmware Frame Format



The final frame within a firmware file contains an embedded reset command that causes the device to reset itself when it has finished updating the application. After resetting, the device performs a CRC on the application, asserts the $\overline{\text{CHG}}$ line and behaves according to incoming commands.

Alternatively, the host may cause the device to reset by sending a frame length word of zero.

If the device is reset part of the way through updating the application, the application will be corrupt and will need updating again.

Note: After updating the firmware the host should rewrite any stored settings used by the application.



Headquarters

Atmel Corporation
2325 Orchard Parkway
San Jose, CA 95131
USA
Tel: 1(408) 441-0311
Fax: 1(408) 487-2600

International

Atmel Asia
Unit 01-05 & 16, 19/F
BEA Tower, Millennium City 5
418 Kwun Tong Road
Kwun Tong
Kowloon
Hong Kong
Tel: (852) 2245-6100
Fax: (852) 2722-1369

Atmel Europe
Le Krebs
8, Rue Jean-Pierre Timbaud
BP 309
78054 Saint-Quentin-en-
Yvelines Cedex
France
Tel: (33) 1-30-60-70-00
Fax: (33) 1-30-60-71-11

Atmel Japan
9F, Tonetsu Shinkawa Bldg.
1-24-8 Shinkawa
Chuo-ku, Tokyo 104-0033
Japan
Tel: (81) 3-3523-3551
Fax: (81) 3-3523-7581

Touch Technology Division
1 Mitchell Point
Ensign Way
Hamble
Southampton
Hampshire SO31 4RF
United Kingdom
Tel: (44) 23-8056-5600
Fax: (44) 23-8045-3939

Product Contact

Web Site
www.atmel.com

Technical Support
touch@atmel.com

Sales Contact
www.atmel.com/contacts

Literature Requests
www.atmel.com/literature

Disclaimer: The information in this document is provided in connection with Atmel products. No license, express or implied, by estoppel or otherwise, to any intellectual property right is granted by this document or in connection with the sale of Atmel products. **EXCEPT AS SET FORTH IN ATMEL'S TERMS AND CONDITIONS OF SALE LOCATED ON ATMEL'S WEB SITE, ATMEL ASSUMES NO LIABILITY WHATSOEVER AND DISCLAIMS ANY EXPRESS, IMPLIED OR STATUTORY WARRANTY RELATING TO ITS PRODUCTS INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT. IN NO EVENT SHALL ATMEL BE LIABLE FOR ANY DIRECT, INDIRECT, CONSEQUENTIAL, PUNITIVE, SPECIAL OR INCIDENTAL DAMAGES (INCLUDING, WITHOUT LIMITATION, DAMAGES FOR LOSS OF PROFITS, BUSINESS INTERRUPTION, OR LOSS OF INFORMATION) ARISING OUT OF THE USE OR INABILITY TO USE THIS DOCUMENT, EVEN IF ATMEL HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.** Atmel makes no representations or warranties with respect to the accuracy or completeness of the contents of this document and reserves the right to make changes to specifications and product descriptions at any time without notice. Atmel does not make any commitment to update the information contained herein. Unless specifically provided otherwise, Atmel products are not suitable for, and shall not be used in, automotive applications. Atmel's products are not intended, authorized, or warranted for use as components in applications intended to support or sustain life.

© 2009 – 2010 Atmel Corporation. All rights reserved. Atmel®, Atmel logo and combinations thereof, QMatrix® and others are registered trademarks, maXTouch™ and others are trademarks of Atmel Corporation or its subsidiaries. Other terms and product names may be registered trademarks or trademarks of others.